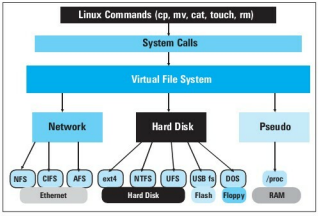

Figure 4: File system interface

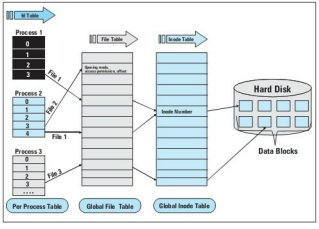

Figure 5: Interaction between processes and VFS objects (normal files)

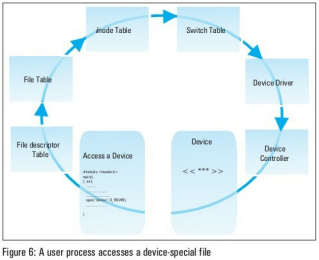

Figure 6: A user process accesses a device-special file

Accessing normal/ordinary files

Linux uses *system calls* to access a file. Let's begin with the *open* system call, which is used to open a file, and which returns a file descriptor for access to the file. This file descriptor is an integer, and its value normally starts at 3, since file descriptors 0, 1 and 2 are assigned for the process' standard input (*stdin*), standard output (*stdout*) and standard error (*stderr*) respectively. Each process has a file descriptor table, in which file descriptors are linked to the file table, and to the inode table, subsequently. The inode table stores a unique inode number for each file—so the total number of inodes in the table is equivalent to the total number of files in the file system. Both the file table and inode table are global tables. The given inode number is routed to a data block on the hard disk/storage device, where the file is stored. An overview of the file-opening operation is shown in Figure 5.

Accessing device-special files

An ordinary file is stored in a hard disk data block, so we can access it easily. But what happens with a device-special file? Since the device file is an entity used to access a device, if you store/write any data to such a file, it will be sent to the device, and not actually stored in a file. Device-special files are in the */dev* directory. Each device file is associated with two numbers -- its major and minor numbers. The major number is used to identify the device type, and the minor number is used to identify the instance of that device type. In this case, the inode table points to a switch table.

There are two switch tables in Linux—character and block device switch tables. Using the major number, the switch table routes to a corresponding device driver, and the device driver accesses the device. The complete flow of a user process accessing a device is shown in Figure 6.

Linux system calls and strace

Linux has approximately 338 system calls. The system call table is stored in *unistd.h (/usr/src/linux-2.6.34/arch/x86/include/asm/unistd_32.h)*, and each system call is identified by a unique system-call number. The file- and file-system-related system calls are given in Table 2, with their call numbers and a brief description of each call.

File system related system calls are available in Table 2 carried in the LFY CD bundled with this issue.

Let's say you want to trace a system call. You want to find out which system calls an executable program uses, how many times each system call is invoked, and how much time it takes for each invocation. We can get all this information from the strace command-line utility. For example, assume I want to create a named pipe, i.e., a FIFO file, to communicate between two processes. To do this, I'd run the *mkfifo* command. We can execute *mkfifo* via *strace* to obtain system call coverage information, as shown in Figures 7 and 8.

Figure 8 shows the execution of strace –c –F *mkfifo*, and lists the information in a different format. Using this, we can see which system calls were used by *mkfifo*, how many times they were used, and how much time each call took. This reveals that *mkfifo* calls *mknod* to create a file; indeed, *mkfifo* is just a wrapper around the *mknod* system call. Thus,